ANALOG DEVICES
AHEAD OF WHAT'S POSSIBLE™

4-Wire SPI Slave Interface with Internal Register Map

Prepared by:

Ahmed Yasser

1. Objective

The goal of this assignment is to design a standard 4-wire SPI slave controller (CSB, SCLK, SDI, SDO) that provides read and write access to an internal register map. The register map is 32768 registers deep and 8 bits wide, addressed by a 15-bit address field. The design must support both single-byte and multi-byte (burst, auto-incrementing) accesses, and must be able to present read data back to the master with no more than half a SCLKperiod of turnaround time after the address is received.

2. SPI Frame Format

Every transaction begins witha16-bit header, transmitted MSB-first, followed by one or more 8-bit data bytes:

Field	Bits	Description
R/W	[15]	0 = Write (master drives data on SDI) 1 = Read (slave drives data on SDO)
ADDRESS	[14:0]	Starting register address; auto-increments on each additional byte during a burst
DATA	8 per byte	Register data, MSB first

A single-byte access there fore takes 24 SCLK cycles (16 header + 8 data); a burst of N bytes takes 16 + 8N cycles, with no dedicated “burst” bit — the slave simply keeps auto-incrementing the address for as long as CSB stays low and SCLK keeps toggling.

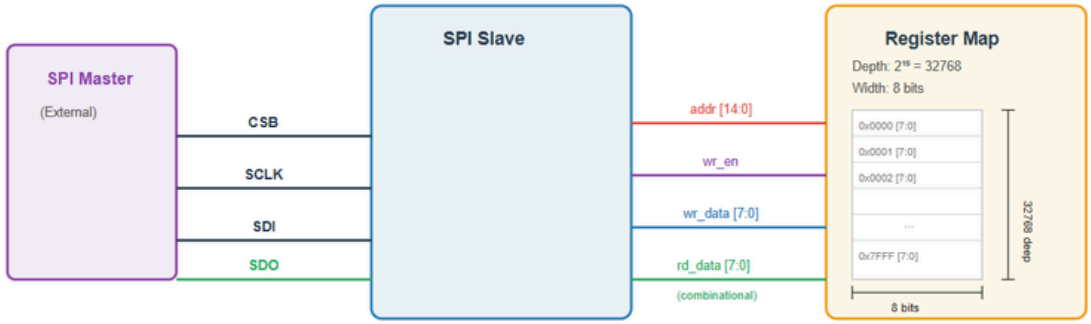
3. Architecture Overview

The design is split into three modules:

- **spi_wrapper** — top-level module exposing only the 4-wire SPI interface (csb, sclk, sdi, sdo). Instantiates spi_slave and reg_map and wires them together through addr / wr_en / wr_data / rd_data. It also implements the SDO tri-state mux (sdo = sdo_en ? sdo_val : 1'bz).
- **spi_slave** — the protocol engine: a finite state machine and shift registers that sample SDI on the rising edge of SCLK, drive SDO on the falling edge, and generate addr / wr_en / wr_data for the register map.
- **reg_map** — the 32768 × 8 register file. Writes are synchronous (one write per completed byte); reads are fully combinational (assign rd_data = mem[addr]), which is what allows read data to be ready within half a SCLK cycle.

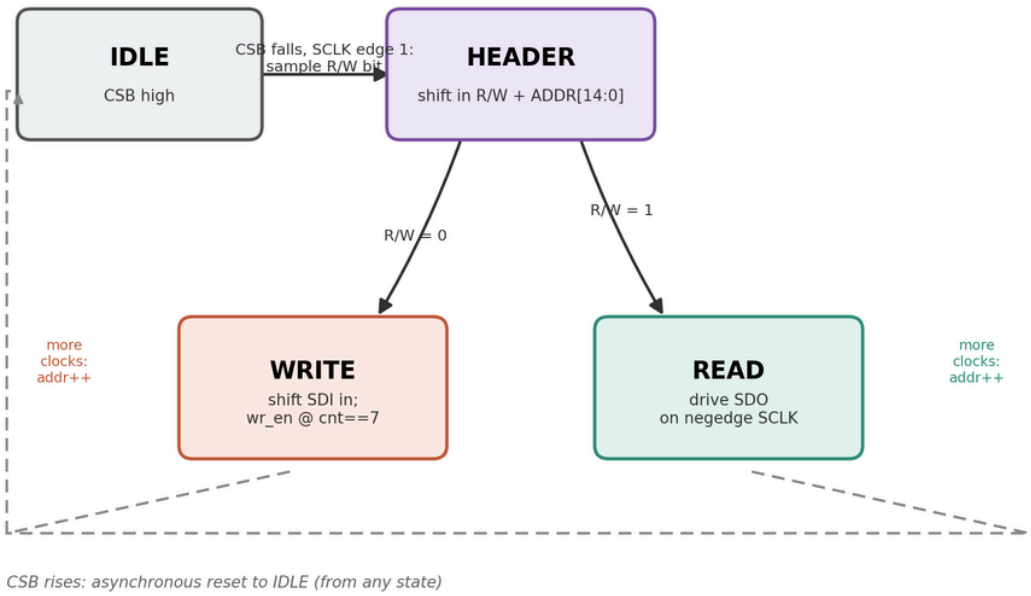
The key design decision that makes the half-clock read-turnaround requirement easy to satisfy is using **opposite clock edges** for input and output: SDI is always sampled on the rising edge of SCLK, and SDO is always driven on the falling edge. Because rd_data is purely combinational off addr, and addr only ever changes on a rising edge, the very next falling edge — half a period later — is guaranteed enough time for the address decode and register read to settle.

CSB is used as an **asynchronous reset** for the FSM and all bit/byte counters: the instant CSB rises, the slave snaps back to the IDLE state.



4. Finite State Machine

The controller usesa 4-state FSM:**IDLE**, **HEADER**, **WRITE** and **READ**. Every SCLK edge does useful work — there are no wasted or “latch-only” cycles, since the master's clock count is fixed at exactly 16 + 8N edges per transaction with no slack.



State	Behaviour
IDLE	Waiting for CSB to fall. The very first SCLK rising edge after CSB goes low samples bit [15] (R/W) and moves to HEADER
HEADER	Shifts in the 15-bit address (A14..A0), MSB first, over the next 15 rising edges.
WRITE	Shifts in one data byte (8 rising edges). wr_en and wr_data are driven combinationally so reg_map captures the write
READ	Tracks bit position for the byte currently being read out. The actual SDO value is driven in a separate falling-edge

5. Module Descriptions

5.1 spi_slave.sv

Implements the FSM described in Section 4. Two clocked processes are used: one on posedge SCLK (or posedge CSB as an asynchronous reset) handling state transitions and bit/byte counting, and one on negedge SCLK driving SDO and its output-enable signal SDO_en. wr_en, wr_data and addr are driven combinationally, not registered, so that reg_map always sees a valid write on the exact clock edge the byte completes.

5.2 reg_map.sv

A 32768 × 8 memory array. Writes happen synchronously on `posedge clk` when wr_en is asserted. Reads are a plain continuous assignment (`assign rd_data = mem[addr]`), making them purely combinational as required by the half-clock read-turnaround specification.

5.3 spi_wrapper.sv

Top-level module. Instantiates spi_slave and reg_map, connects them through the internal addr / wr_en / wr_data / rd_data bus, and tri-states SDO whenever SDO_en is low (: 1'bz), so the bus is correctly released outside of read-data phases.

6. Testbench

- `spi_write(addr, data) — single-byte write`
- `spi_read(addr, data) — single-byte read`
- spi_write_burst(addr, data) — 4-byte burst write, CSB held low throughout
- spi_read_burst(addr, data) — 4-byte burst read, CSB held low throughout

Every check goes through a check_byte task that compares the captured value against the expected one, printing PASS/FAIL and incrementing either correct_count or error_count — making the testbench self-checking rather than requiring manual waveform inspection.

Test cases covered:

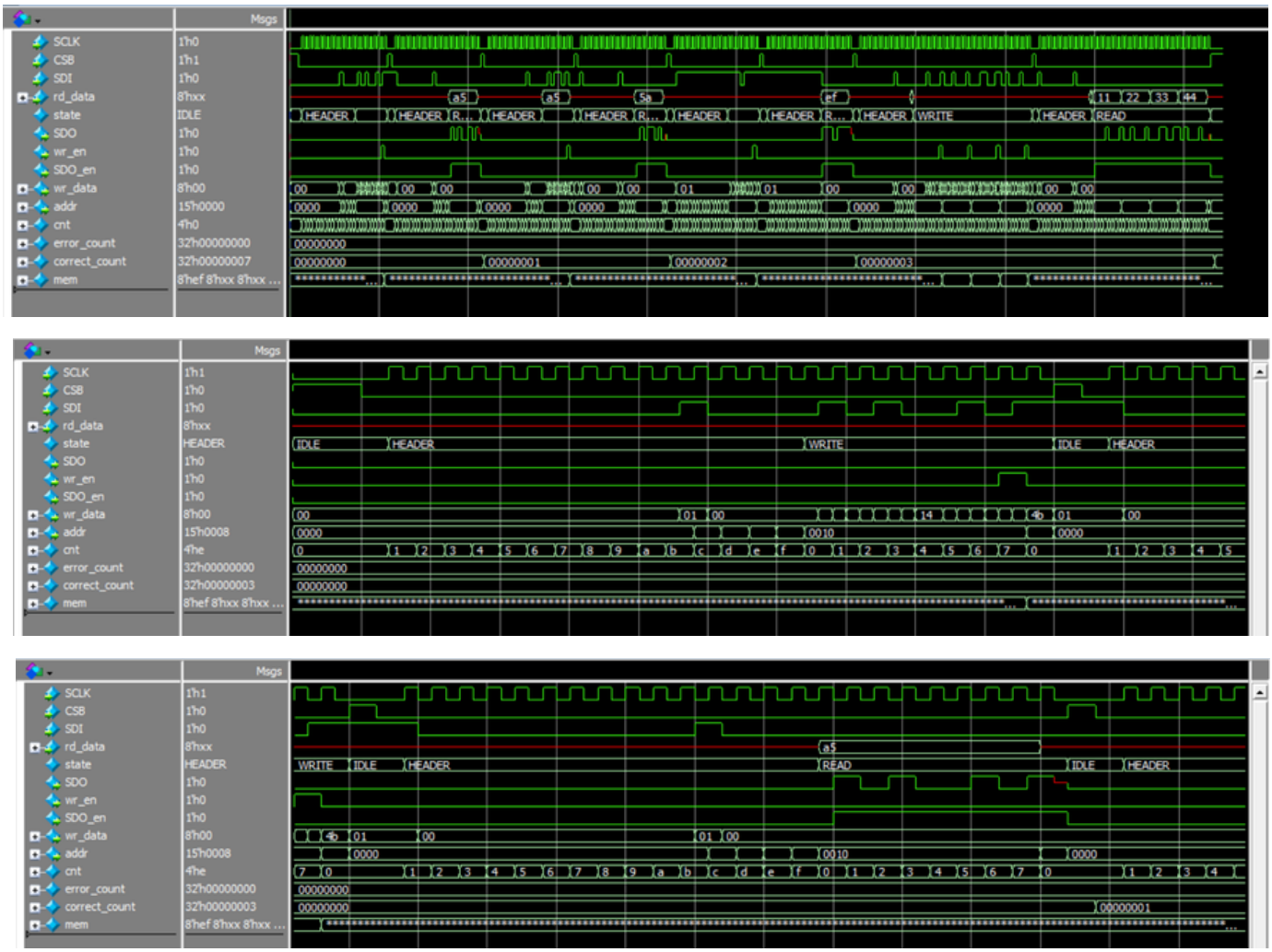
- Single-byte write/read at address 0x0010
- Overwrite: write a new value to the same address and confirm the old value is gone
- 4-byte burst write, then 4-byte burst read, verifying auto-increment addressing

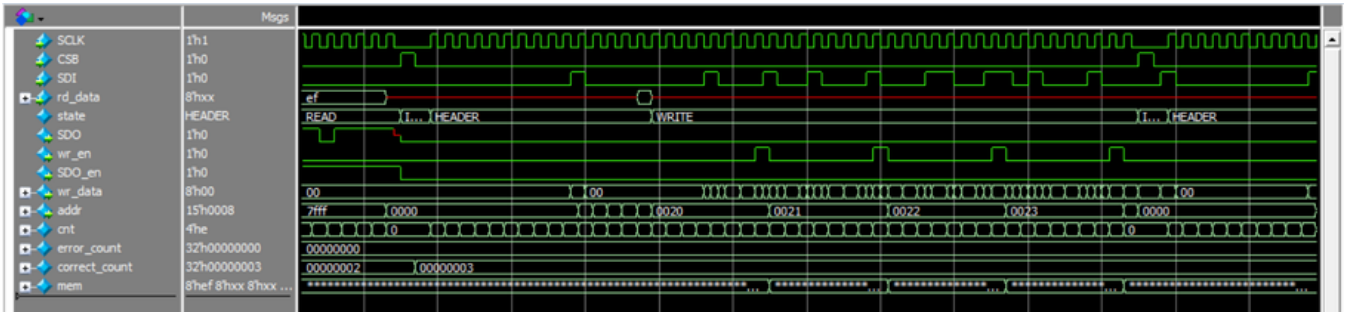
7. Simulation Results

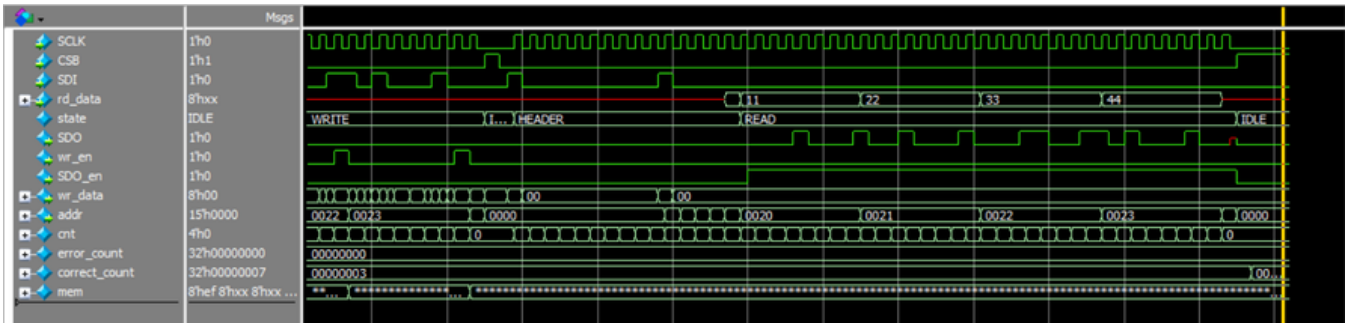
The design was simulated in QuestaSim. The transcript below confirms all seven self-checks passed with zero errors:

```
# ----------------------------------------------------------
# Start simulation
# ----------------------------------------------------------
# 50 : single-byte write/read
# PASS: addr 0x0010 readback  = 0xa5
# 1090 : overwrite
# PASS: addr 0x0010 after overwrite  = 0x5a
# PASS: addr 0x7FFF readback  = 0xef
# 3170 : burst write/read
# PASS: burst byte 0 (addr 0x0020)  = 0x11
# PASS: burst byte 1 (addr 0x0021)  = 0x22
# PASS: burst byte 2 (addr 0x0022)  = 0x33
# PASS: burst byte 3 (addr 0x0023)  = 0x44
# ----------------------------------------------------------
# simulation Finshed
# ----------------------------------------------------------
# correct count = 7
# error count = 0
# ----------------------------------------------------------
```

The waveform below shows the FSM stepping through HEADER, WRITE and READ states across the test sequence, along with wr_en pulsing exactly once per completed write byte, and the mem[] array updating for the burst-write test (addresses 0x0020–0x0023 receiving 0x11 / 0x22 / 0x33 / 0x44):




Msgs
SCLK
CSB
SDI
rd_data
state
SDO
wr_en
SDO_en
wr_data
addr
cnt
error_count
correct_count
mem
1'h1
1'h0
1'h0
8'hxx
HEADER
1'h0
1'h0
1'h0
8'h00
15'h0008
4'he
32'h00000000
32'h00000003
8'hef 8'hxx 8'hxx ...
ef
READ
HEADER
WRITE
HEADER
00
7fff
0000
0020
0021
0022
0023
0000
0
00000000
00000002
00000003


Msgs
SCLK
CSB
SDI
rd_data
state
SDO
wr_en
SDO_en
wr_data
addr
cnt
error_count
correct_count
mem
1'h0
1'h1
1'h0
8'hxx
IDLE
1'h0
1'h0
1'h0
8'h00
15'h0000
4'h0
32'h00000000
32'h00000007
8'hef 8'hxx 8'hxx ...
11
22
33
44
WRITE
HEADER
READ
IDLE
00
0022
0023
0000
0020
0021
0
00000000
00000003